Equal Work Distribution Matrix: HTML, CSS, JavaScript & Python

Project: Hospital Patient Queue, Bed Availability & Admission Management System
Course: Advanced Programming (AP) Project
Team Size: 3 Members
Distribution Principle: Strictly Equal 33.3% / 33.3% / 33.3% share across **HTML**, **CSS**, **JavaScript (React & Node.js)**, and **Python (FastAPI & Analytics)**.

---

1. Executive Summary & Verification Matrix

To ensure compliance with academic evaluation standards, no single member is concentrated solely in frontend design, backend infrastructure, or data science. Every teammate writes an equal, demonstrable share of code across all 4 foundational technologies:

Technology Layer	Person A: Ishika (Beds & Admissions)	Person B: Aarpan (OPD Queue & Triage)	Person C: Chinmaya (Platform, Auth & Regional View)	Total Balance
HTML (Markup & Semantics)	33.3%	33.3%	33.3%	**100% Equal**
CSS (Tokens, Layouts & Motion)	33.3%	33.3%	33.3%	**100% Equal**
JavaScript (React UI & Node API)	33.3%	33.3%	33.3%	**100% Equal**
Python (Analytics & Algorithms)	33.3%	33.3%	33.3%	**100% Equal**

---

2. Deep-Dive Division by Technology

2.1. Python (`analytics/` Microservice)

Previously, Python analytics was concentrated in queueing math. To guarantee **equal Python contribution**, the Python analytics service is partitioned into **three distinct algorithmic domain engines**, each featuring its own data models, algorithmic logic, FastAPI endpoints, and Pytest test suites:

```
analytics/
├── main.py                    # Microservice App Assembly & Router Mounting (Person C)
├── requirements.txt
├── routers/
│   ├── bed_router.py          # Inpatient Analytics Routes (Person A)
│   ├── queue_router.py        # OPD Queue Analytics Routes (Person B)
│   └── city_router.py         # Regional Surge & Referral Routes (Person C)
├── core/
│   ├── bed_analytics.py       # Length of Stay & Bed Depletion Models (Person A)
│   ├── queue_engine.py        # Erlang-C & Multi-Server Queue Math (Person B)
│   └── city_analytics.py      # Strain Index & Optimization Routing (Person C)
└── tests/
    ├── test_bed_analytics.py  # Pytest suite for Inpatient Models (Person A)
```

```
    ├── test_queue_engine.py    # Pytest suite for Erlang-C Math (Person B)
    └── test_city_analytics.py   # Pytest suite for Regional Optimization (Person C)
```

Detailed Python Work Allocation:

1. **Person A: Ishika (Inpatient & Bed Depletion Modeling)**
2. **Core File:** `analytics/core/bed_analytics.py` & `analytics/routers/bed_router.py`
3. **Algorithms & Models:**
    - **Average Length of Stay (ALOS) & Turnover Modeling:** Calculates ward-level turnover rate and expected stay durations modeled via log-normal/Poisson arrival-stay functions.
    - **Ward Depletion & Surge Forecast:** Predicts time-to-zero bed exhaustion based on admission inflow rates and scheduled discharge cohorts.
4. **Endpoints:**
    - `POST /analytics/beds/forecast`
    - `POST /analytics/beds/turnover`
5. **Automated Tests:** `analytics/tests/test_bed_analytics.py` (Tests zero-capacity limits, high turnover variance, edge cases where discharge rate $\ge$ admission rate).
6. **Person B: Aarpan (OPD Queueing & Erlang-C Modeling)**
7. **Core File:** `analytics/core/queue_engine.py` & `analytics/routers/queue_router.py`
8. **Algorithms & Models:**
    - **Multi-Server Queueing ($M/M/c$) via Erlang-C:** Exact calculation of probability of waiting $C(c, a)$, expected waiting time $W_q = \frac{C(c, a)}{c\mu - \lambda}$, and percentile wait-time confidence bands.
    - **Dynamic Triage Priority Offsets:** Mathematically ages queue priorities so urgent cases jump ahead while non-urgent cases do not encounter unbounded starvation.
9. **Endpoints:**
    - `POST /analytics/queue/estimate`
    - `POST /analytics/queue/saturation`
10. **Automated Tests:** `analytics/tests/test_queue_engine.py` (Tests saturation boundary $\rho \ge 1$, single vs multi-doctor convergence, zero-doctor exceptions).
11. **Person C: Chinmaya (Regional Healthcare Strain & Referral Optimization)**
12. **Core File:** `analytics/core/city_analytics.py`, `analytics/routers/city_router.py`, `analytics/main.py`
13. **Algorithms & Models:**
    - **City-Wide Healthcare Strain Index (HSI):** Multi-factor weighted composite scoring model combining ICU utilization, emergency bed saturation, and average OPD queue lag across city hospitals.
    - **Optimal Patient Referral Routing Algorithm:** Distance-weighted capacity optimization that calculates best target hospital for patient transfers to minimize mortality risk and regional load imbalance.
14. **Endpoints:**
    - `POST /analytics/city/strain`
    - `POST /analytics/city/recommend-referral`
    - `GET /health` (Service health & runtime configuration verification)
15. **Automated Tests:** `analytics/tests/test_city_analytics.py` (Tests weighted strain matrix, zero-bed edge cases across multiple facilities, outlier smoothing).

2.2. HTML (Semantic Markup & JSX UI Structure)

Markup responsibility is divided cleanly by operational boundary, giving each student ownership of critical interactive templates and accessibility structures:

1. **Person A: Ishika (Inpatient Wards & Admission Markup)**
2. **Ward Bed Map Grid:** Semantic grid markup for ward floorplans, bed cards (`BedCard.jsx`), isolation units, and equipment indicator badges.
3. **Inpatient Workflow Modals:** Accessible `<dialog>` markup for Patient Admission, Bed Transfer, and Patient Discharge workflows.
4. **Housekeeping Checklist:** Semantic table and form controls for ward sanitation tracking and bed turn-around clearance.
5. **Person B: Aarpan (OPD Queue, Doctor Console & Public Kiosk Markup)**
6. **Doctor Calling Console:** Semantic tables and calling controls (Call Next, Hold, Escalate, Mark No-Show).
7. **Public Waiting-Room Display:** High-visibility TV display markup for public waiting areas (strictly token numbers, zero PHI, high contrast semantic tags).
8. **Digital Token Slip:** Accessible token pass markup featuring QR placeholder, department, priority status, and "Symptoms Worsened" emergency button.
9. **Person C: Chinmaya (Application Shell, Security & Regional Directory Markup)**
10. **Global App Shell:** Semantic `<header>`, `<nav>`, responsive drawer navigation, user profile badge, and breadcrumbs (`AppShell.jsx`).
11. **Authentication Markup:** Accessible login form, password visibility controls, account lockout countdown alert banner, and role selection screens.
12. **Master Patient Registration:** Demographic intake form with accessible validation messages, UHID search inputs, and duplicate patient detection modal.
13. **Regional Overview Layout:** Multi-hospital capacity comparison cards and aggregated metric tiles.

2.3. CSS (Design Tokens, Modular Stylesheets & Animations)

CSS is split into three standalone stylesheets plus a shared token system, preventing merge conflicts while guaranteeing equal styling effort:

```
web/src/styles/
├── tokens.css          # Design Tokens, Color Palettes, Typography (Shared Foundation – Person C)
├── layout.css          # App Shell, Navigation, Global Form Inputs (Person C)
├── beds.css            # Ward Maps, Bed Cards, Occupancy Badges & Modals (Person A)
└── queue.css           # Queue Boards, Public TV Kiosks, Audio Wave Animations (Person B)
```

1. **Person A: Ishika (`beds.css`)**
2. **CSS Grid Ward Map:** Auto-fit responsive grid adapting from 320px mobile viewports to ultra-wide displays without card squishing.
3. **Occupancy Badges & Micro-Animations:** Color-coded status badges with gentle pulse animations for `OCCUPIED`, `AVAILABLE`, `CLEANING`, and `RESERVED`.
4. **Modal Transition Keyframes:** Smooth backdrop-filter blur and slide-up modal animation keyframes (`@keyframes slideUp`).
5. **Person B: Aarpan (`queue.css`)**

6. **High-Contrast Public TV Theme:** WCAG AAA compliant styling for public displays with glare-resistant dark background and ultra-readable high-contrast typography.
7. **Active Token Pulse & Alert Flash:** CSS keyframes for flashing urgency when an emergency triage token is escalated.
8. **Audio Visualizer Indicator:** CSS animation simulating sound waves when a token number is called over the PA system.
9. **Person C: Chinmaya (`tokens.css`, `layout.css`, `forms.css`)**
10. **Design Tokens Architecture:** Root CSS variables (`:root`) defining elevation shadows, radius scales, font hierarchy, and light/dark theme variables.
11. **Responsive Layout Shell:** CSS Flexbox/Grid layouts for sidebar toggle, sticky top-nav, and main scroll containers.
12. **Form States & Error Animations:** Floating labels, input focus rings, disabled button styling, and shake animation (`@keyframes shake`) on authentication errors.

2.4. JavaScript (Full-Stack: React Hooks & Express API Backend)

JavaScript represents the core runtime of the application and is split across frontend React state logic and backend Node.js / Express controllers:

```
Frontend React (web/src/)              Backend Node.js (api/src/)
├── hooks/                             ├── controllers/
│   ├── useBedManagement.js (A)        │   ├── bed.controller.js (A)
│   ├── useQueue.js (B)                │   ├── queue.controller.js (B)
│   └── useAuth.jsx (C)                │   ├── auth.controller.js (C)
├── components/                        │   └── patient.controller.js (C)
│   ├── beds/ (A)                      ├── services/
│   ├── queue/ (B)                     │   ├── bed.service.js (A)
│   └── auth/ & city/ (C)              │   ├── queue.service.js (B)
│                                      │   ├── auth.service.js (C)
│                                      │   └── patient.service.js (C)
```

1. **Person A: Ishika (Inpatient Logic & Concurrency Control)**
2. **Frontend JS (React):**
   - `useBedManagement.js`: Hook managing bed data fetching, optimistic UI updates for bed status, and ward filtering.
   - Bed allocation workflow handlers (intake validation, bed assignment triggers, transfer confirmation).
   - Countdown timer state logic for expiring temporary bed reservations.
3. **Backend JS (Node.js):**
   - `bed.controller.js` & `bed.service.js`: REST endpoints for beds, wards, transfers, and discharges.
   - Concurrency Transaction Engine: PostgreSQL row-locking implementation via `withTransaction()` utilizing `SELECT ... FOR UPDATE SKIP LOCKED` to prevent double-booking.
   - Event Listener: Subscribes to `discharge` events to initiate housekeeping transitions.
4. **Person B: Aarpan (Queue State, Polling & Triage Transitions)**
5. **Frontend JS (React):**
   - `useQueue.js` & `usePolling.js`: Real-time polling hook with exponential backoff, stale-data warnings, and window focus revalidation.

- Doctor console actions: Next token dispatching, no-show counters, and doctor pause/resume timers.
- "Symptoms Worsened" client escalation handler with instantaneous optimistic priority elevation.

6. **Backend JS (Node.js):**
    - `queue.controller.js` & `queue.service.js`: Endpoints for joining queues, calling tokens, and viewing queue depths.
    - Atomic Token Sequencer: Safe token sequence generation using atomic database counters.
    - Compare-and-Swap Transitions: Safe state updates (`WHERE status = 'WAITING'`) preventing simultaneous token assignments.
    - Event Emitter: Emits `patientCalled` and `symptomsEscalated` events across the central system bus.

7. **Person C: Chinmaya (Security, Auth Pipeline, Patient Core & API Infrastructure)**

8. **Frontend JS (React):**
    - `useAuth.jsx` & `AuthContext`: Token storage in secure memory, JWT expiration checking, and automatic route redirection.
    - Protected Route Guards: `RequireAuth` and `RequireRole` components enforcing hospital-scoped access control.
    - Debounced Patient Search: Client-side search with debounce to prevent duplicate registration requests.
9. **Backend JS (Node.js):**
    - Express Pipeline Assembly: `app.js` and `server.js` configuring 9-stage middleware pipeline (`helmet`, `cors`, `pino-http`, rate limiting, centralized error handling).
    - `auth.controller.js` & `auth.service.js`: Argon2id password hashing, 5-attempt brute-force lockout, and JWT token signing.
    - `patient.controller.js` & `patient.service.js`: Unique UHID generator and duplicate patient matching logic.
    - Idempotency Replay Middleware: Caching and deterministic replay of POST requests using UUID idempotency keys.

---

3. Academic Verification & Git Commit Strategy

To demonstrate this equal work division in git commit histories and code audits:

1. **Branch Scoping:**
2. `feat/beds-ishika`: Commits touching `beds.css`, `bed_analytics.py`, `bed.controller.js`, `BedCard.jsx`.
3. `feat/queue-aarpan`: Commits touching `queue.css`, `queue_engine.py`, `queue.controller.js`, `QueueBoard.jsx`.

4. `feat/platform-chinmaya`: Commits touching `tokens.css` / `layout.css`, `city_analytics.py`, `auth.controller.js`, `AppShell.jsx`.

5. **Git Statistics Target per Member:**

6. ~33% of commits in `/analytics/` (Python)
7. ~33% of commits in `/web/src/styles/` (CSS)
8. ~33% of commits in `/web/src/components/` (HTML/JSX)
9. ~33% of commits in `/api/` & `/web/src/hooks/` (JavaScript)

This structure ensures total technical rigor, zero freeloading risk, and clear accountability for examination vivas and code reviews.